

HASHFORGE PRO

Interactive Cryptographic & Malware Analysis Laboratory

Project Research Paper and Presenter Guide

Academic context: TY B.Sc. CDS project
Prepared for college faculty and beginner learners

Abstract

HashForge Pro is a browser-based educational laboratory for learning how hash functions support cybersecurity and digital forensics. The application lets students calculate real SHA-256, SHA-384, and SHA-512 digests, build Merkle Trees, compare HMAC with a plain secret-prefix hash, study output distributions, explore PBKDF2, compare synthetic near-duplicate samples, calculate collision probabilities, and check local file integrity. The project focuses on safe interaction, simple explanations, and technical honesty. It does not upload files, execute malware, or claim that classroom demonstrations prove real-world security.

Keywords: cryptographic hash, Merkle Tree, HMAC, PBKDF2, fuzzy hashing, file integrity, digital forensics

1. Introduction

Hash functions are used in many security systems, but beginners often meet them as static definitions and diagrams. A student may know that a hash creates a digest but still find it difficult to see why one small input change creates a very different result, how several records become one Merkle Root, or why password storage needs a slower method.

HashForge Pro addresses this learning gap through small browser-side experiments. Each laboratory follows the same pattern: enter safe data, perform a real calculation, observe a visual result, and connect that result to a practical security use case.

2. Problem Statement

Students can read about hashing, but they cannot always see its behavior. Static learning makes it harder to connect hashing with avalanche behavior, integrity checking, password security, malware analysis, and collision probability. A practical learning tool should let students change inputs and receive immediate feedback while keeping the examples safe.

3. Objectives

- Make common hash properties visible through interaction.
- Connect cryptographic ideas to file integrity and forensics.
- Use real browser-side calculations instead of fake demonstrations.
- Give students a safe environment with synthetic data and local processing.
- Provide a presentation flow that a student can explain to faculty.

4. System Design and Privacy

The application uses React, TypeScript, Vite, the Web Crypto API, Recharts, and browser storage. Cryptographic calculations run locally. Selected files are processed in the browser and are not uploaded. Experiment History stores only non-sensitive metadata. Presentation edits are saved locally and shared between the Presentation Editor and Live Presentation.

The tool uses controlled examples for advanced topics. The HMAC laboratory compares a vulnerable construction with HMAC without attacking a real service. The fuzzy hashing laboratory is explicitly a simulator, not the official ssdeep algorithm. The malware-analysis content uses synthetic samples and does not execute malware.

5. Implemented Laboratories

Hash Explorer

Calculates SHA-256, SHA-384, or SHA-512 for text. The user clicks Generate digest and can compare Hello with hello. The fixed digest size and changed output demonstrate deterministic hashing and the fingerprint idea.

Avalanche Effect

Live Presentation compares Hello World with Hello World! and shows both real SHA-256 digests plus a changed-bits meter. The audience can see that a small input change creates a large output change.

Merkle Tree

Merkle Tree Lab accepts four editable blocks and recalculates leaf hashes, parent hashes, and the Merkle Root. Changing one block demonstrates how the change travels to the root.

HMAC vs Plain Hash

The user enters a toy secret and message, then clicks Compare constructions. The tool shows a simple secret-prefix hash beside HMAC and labels the simple construction as insecure.

Entropy and Distribution

The user chooses an input count and pattern, then clicks Run experiment. The tool shows a byte histogram, sample entropy, average Hamming distance, and a bit ratio. These measurements illustrate output behavior but are not a security proof.

Password Hashing / PBKDF2

The user enters a demo password, generates a salt, changes the iteration slider, and clicks Run derivation. The tool compares fast hashing with repeated PBKDF2 work and explains why salts matter.

Fuzzy Hashing

Two synthetic samples are compared with a chunk-overlap similarity estimate. Editing Sample B changes the live percentage. The interface clearly says that this is a simulator, not official ssdeep.

Birthday Collision

The user selects a small digest space such as 8-bit, moves the Samples slider, and observes Collision chance, the 50% bound, and the probability graph. Large hash sizes use mathematical approximation rather than brute force.

File Integrity / Forensics

The user clicks Choose local file, sees SHA-256 and SHA-512 fingerprints, and clicks Create baseline. Selecting the same or changed version produces UNCHANGED or MODIFIED. Processing stays local.

Malware Analysis Use Case

The project presents a safe workflow: suspicious file, SHA-256, hash comparison, known information, and further investigation. This is a synthetic use case, not a malware scanner. A hash helps identify a sample but does not prove that it is malicious.

Presentation Mode

The faculty-facing Live Presentation follows problem, importance, solution, demos, applications, benefits, future scope, and conclusion. It supports arrow-key navigation, fullscreen, presenter notes, editing, and locally saved presentation content.

6. Presenter Demonstration Script

Opening

Good morning. Our project is HashForge Pro, an interactive cryptographic and malware-analysis laboratory. We created it because hashing is easy to read about but difficult to understand without seeing the results.

Problem and solution

Students usually see slides and static diagrams. HashForge Pro lets them enter data, run real browser-side calculations, and see what changes. The examples are safe, local, and simple enough to explain during a viva.

Live hash demo

I will enter Hello and generate its SHA-256 digest. Then I will change one letter and generate again. The output changes completely, which shows why a hash can work like a fingerprint for exact data.

Live Merkle demo

These four blocks are hashed into a tree. I will change Block B and recalculate it. The change moves through the parent hashes and produces a different Merkle Root.

Security uses

The same idea helps check file integrity and compare samples during forensic work. A hash match is useful evidence, but it does not prove that a file is safe or malicious by itself.

Closing

HashForge Pro turns abstract cryptography into experiments students can see and explain. Our future scope is to add more algorithms, forensic activities, learning challenges, and classroom controls while keeping the workflow simple.

7. Educational Benefits

- Students see how a hash reacts to a small input change.
- Students connect hash concepts to file integrity and forensics.
- Students compare exact hashes with fuzzy similarity.
- Students understand why password hashing needs salts and extra work.
- Students can experiment without real malware or credential attacks.
- Students can practice explaining technical results in simple English.

8. Limitations and Technical Accuracy

- Hashing is not encryption.
- SHA-256 is a general-purpose hash, not a password-storage method by itself.
- Collisions are possible in theory, even when they are difficult to find.
- Entropy graphs show sample behavior, not a cryptographic proof.
- Fuzzy similarity is not the same as an exact cryptographic hash.
- A hash match checks the compared bytes but does not prove that a file is safe.
- The project uses harmless synthetic examples and does not execute real malware.

9. Future Scope

Reasonable future additions include more algorithms, more forensic experiments, more learning challenges, teacher classroom controls, offline support, and additional cybersecurity laboratories. These additions can build on the existing browser-first structure without requiring a backend.

10. Conclusion

HashForge Pro combines real browser-side cryptographic calculations with simple explanations and visual experiments. It helps beginners move from reading about hashing to observing its behavior. The project is useful as a learning aid, a classroom demonstration, and a safe introduction to practical security workflows.

References

- NIST, Secure Hash Standard, FIPS 180-4.
- NIST, Recommendation for Password-Based Key Derivation, SP 800-132.
- W3C, Web Cryptography API.
- HashForge Pro source code and implemented browser laboratories.